Лабораторная работа №6.
Работа с интерфейсами связи. SPI. I2C.

Цель работы: освоиться с принципами работы интерфейсов SPI и I2C и научиться программировать их работу, используя функции HAL.

Общие сведения:

SPI (Serial Peripheral Interface) — это синхронный последовательный протокол передачи данных, используемый для обмена информацией между микроконтроллерами и периферийными устройствами, такими как датчики, дисплеи, SD-карты, Flash-память и другие. SPI является одним из самых популярных протоколов благодаря своей простоте и высокой скорости передачи данных.

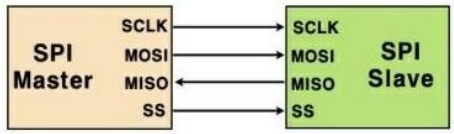
Основные характеристики SPI:

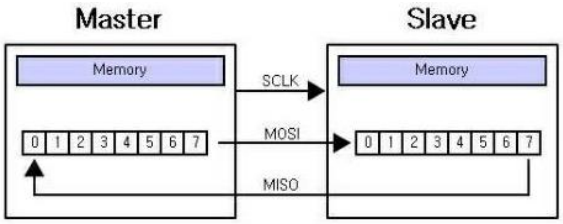
1. **Синхронная передача данных**: SPI использует тактовый сигнал (CLK) для синхронизации передачи данных.
2. **Полный дуплекс**: Данные могут передаваться одновременно в обе стороны (от мастера к слейву и от слейва к мастеру).
3. **Высокая скорость**: SPI поддерживает высокие скорости передачи данных, часто до десятков мегабит в секунду.
4. **Простота реализации**: Протокол требует минимального количества сигналов (обычно 4 линии).

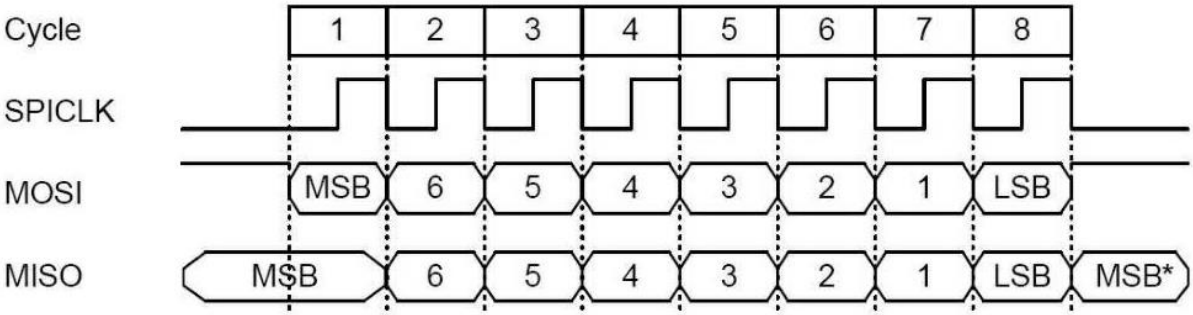
Основные сигналы SPI:

1. **SCLK (Serial Clock)**: Тактовый сигнал, генерируемый мастером. Синхронизирует передачу данных.
2. **MOSI (Master Out Slave In)**: Линия передачи данных от мастера к слейву.
3. **MISO (Master In Slave Out)**: Линия передачи данных от слейва к мастеру.
4. **SS/CS (Slave Select/Chip Select)**: Сигнал выбора слейва. Активен низким уровнем (обычно 0). Мастер использует этот сигнал для выбора конкретного слейва.

Ниже представлена схема передачи данных по SPI.







Передача осуществляется пакетами. Длина пакета, как правило, составляет 1 байт (8 бит), при этом известны реализации SPI с иной длиной пакета, например, 4 бита. Ведущее устройство инициирует цикл связи установкой низкого уровня на выводе выбора подчиненного устройства (SS) того устройства, с которым необходимо установить соединение. При низком уровне сигнала SS:

- схемотехника ведомого устройства находится в активном состоянии;
- вывод MISO переводится в режим «выход»;
- тактовый сигнал SCLK от ведущего устройства воспринимается ведомым и вызывает считывание на входе MOSI значений передаваемых от ведущего битов и сдвиг регистра ведомого устройства.

Подлежащие передаче данные ведущее и ведомое устройства помещают в сдвиговые регистры. После этого ведущее устройство начинает генерировать импульсы синхронизации на линии SCLK, что приводит к взаимному обмену данными. Передача данных осуществляется бит за битом от ведущего по линии MOSI и от ведомого по линии MISO. Передача осуществляется, как правило, начиная со старших битов, но некоторые производители допускают изменение порядка передачи битов программными методами. После передачи каждого пакета данных ведущее устройство, в целях синхронизации ведомого устройства, может перевести линию SS в высокое состояние.

Принцип работы SPI:

1. Мастер генерирует тактовый сигнал (SCLK).
2. Мастер активирует сигнал SS/CS для выбора слейва.
3. Данные передаются одновременно по линиям MOSI и MISO:
    - Мастер отправляет данные на слейв через MOSI.

    - Слейв отправляет данные на мастер через MISO.
4. Передача данных происходит бит за битом, на каждом такте SCLK.
5. После завершения передачи, мастер деактивирует сигнал SS/CS.

Режимы работы SPI:

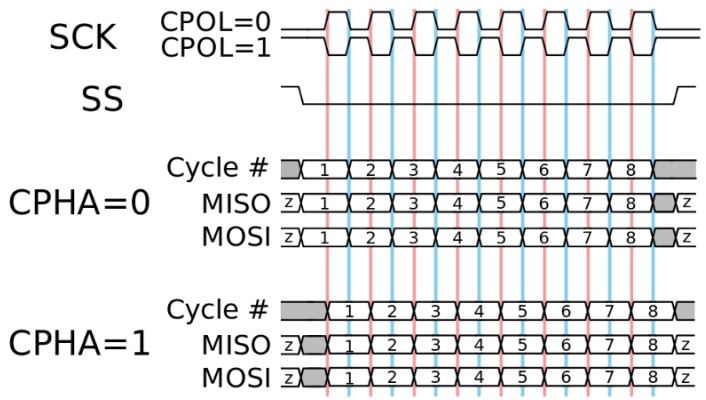
SPI поддерживает 4 режима работы, которые определяются полярностью (CPOL) и фазой (CPHA) тактового сигнала:

- **CPOL (Clock Polarity)**:
    - 0: Тактовый сигнал (сигнал синхронизации) начинается с низкого уровня.
    - 1: Тактовый сигнал (сигнал синхронизации) начинается с высокого уровня.
- **CPHA (Clock Phase)**:
    - 0: Данные сэмплируются (проводится их выборка) на переднем фронте тактового сигнала.
    - 1: Данные сэмплируются (проводится их выборка) на заднем фронте тактового сигнала.

Режимы сигналов SPI:

- Режим 0: CPOL = 0, CPHA = 0
- Режим 1: CPOL = 0, CPHA = 1
- Режим 2: CPOL = 1, CPHA = 0
- Режим 3: CPOL = 1, CPHA = 1

Пример передачи данных по SPI, в зависимости от режимов сигналов, представлен на рисунке ниже.

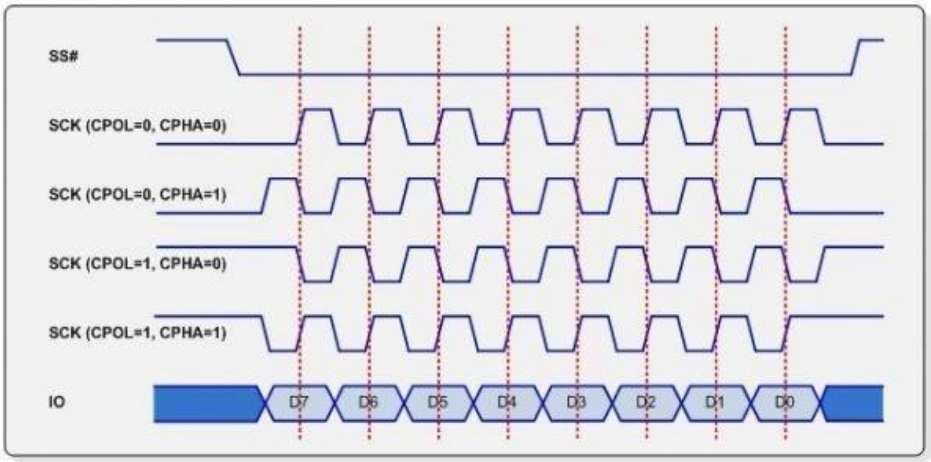


С одной стороны, работа SPI на первый взгляд может показаться сложной, но на самом деле для понимания стоит понимать следующие тезисы:

1) SPI работает по топологии ведущий-ведомый (master/slave). Один задает работу, другой отдает данные.

2) На передачу данных ориентированы два провода: MOSI и MISO
3) SCK нужен для продвижения данных и момент их получения, т.е. от него зависит скорость работы SPI
4) SS необходим для выбора ведомого, с которым будет общаться ведущий
5) CPOL определяет, с какого уровня будет начинаться сигнал синхронизации SCK (0 - с нуля, 1 - с единицы)
6) CPHA определяет, по какому фронту SCK будут браться данные.

Вот еще одна иллюстрация того, как эти режимы SPI работают. По сути, контакты CPOL и CPHA просто определяют, на каком фронте, и на каком импульсе будут браться данные.



Стоит отметить, что использование этих режимов также определяется по даташитам на устройства, которые работают по SPI.

Синхронизация в SPI

Частота следования битовых интервалов в линиях передачи данных определяется синхросигналом SCK, который генерирует ведущее устройство. Ведомые устройства используют синхросигнал для определения моментов изменения битов на линии данных, при этом ведомые устройства никак не могут влиять на частоту следования битовых интервалов. Как в ведущем устройстве, так и в ведомом устройстве имеется счетчик импульсов синхронизации (битов). Счетчик в ведомом устройстве позволяет последнему определить момент окончания передачи пакета. Счетчик сбрасывается при выключении подсистемы SPI, такая возможность всегда имеется в ведущем устройстве. В ведомом устройстве счетчик обычно сбрасывается деактивацией интерфейсного сигнала SS.

Так как действия ведущего и ведомого устройства тактируются одним и тем же сигналом, то к стабильности этого сигнала не предъявляется никаких требований, за исключением ограничения на длительность полупериодов, которая определяется максимальной рабочей частотой более медленного устройства. Это позволяет

использовать SPI в системах с низкостабильной тактовой частотой, а также облегчает программную эмуляцию ведущего устройства.

Регистры SPI

SPI_CR1

Регистр управления SPI_CR1. Задает настройки передачи по SPI.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BIDI MODE	BIDI OE	CRC EN	CRC NEXT	DFF	RX ONLY	SSM	SSI	*LSB FIRST*	SPE	BR [2:0]			MSTR	CPOL	CPHA
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Бит 15. BIDIMODE(Bidirectional data mode enable) — если записать в этот бит единицу данные будут передаваться в двух направлениях по одной линии(однопроводной режим), если ноль —по двум линиям. При двунаправленной передаче master использует вывод MOSI, а slave вывод MISO.

Бит 14. BIDIOE(Output enable in bidirectional mode) — этот бит используется для настройки однопроводного режима, 1 — данные только передаются, 0 — данные только принимаются.

Бит 13. CRCEN(Hardware CRC calculation enable) — запись единицы в этот бит, разрешает аппаратный подсчёт CRC, 0 – запрещает.

Бит 12. CRCNEXT(CRC transfer next) – если этот бит установлен, это говорит о том следующей будет передаваться контрольная сумма.

Бит 11. DFF(Data frame format) – этот бит определяет формат фрейма, 0 — 8 бит, 1 — 16 бит.

Бит 10. RXONLY(Receive only) – этот бит используется в двухпроводном режиме, его установка в единицу запрещает передачу и модуль SPI работает только на приём, иначе на приём и передачу.

Бит 9. SSM(Software slave management) — программное управление NSS, когда этот бит установлен вместо уровня на входе NSS контролируется состояние бита SSI, если сброшен — контролируется состояние вывода NSS.

Бит 8. SSI(Internal slave select) — этот бит имеет силу только при установленном SSM, его значение заменяет состояние на выводе NSS.

Бит 7. LSBFIRST(Frame format) — определяет формат фрейма, 1 — младшим битом вперёд, 0 — старшим битом вперёд.

Бит 6. SPE(SPI enable) — единица в этом бите включает модуль SPI, 0 – выключает.

Бит 5-3. BR[2:0](Baud rate control) — данные биты определяют скорость передачи данных.

Бит 2 MSTR(Master selection) — определяет режим работы, 1 —Master, 0 — Slave.

Бит 1 CPOL(Clock polarity) — определяет полярность в режиме ожидания, 0 — низкий уровень в режиме ожидания, 1 — высокий уровень в режиме ожидания.

Бит 0 CPHA(Clock phase) – задаёт фазу тактового сигнала, 0 — выборка данных производится по переднему фронту сигнала синхронизации, 1 — выборка данных производится по заднему фронту сигнала синхронизации.

SPI_CR2

Регистр управления SPI_CR2. Задает настройки передачи по SPI.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								TXEIE	RXNEIE	ERRIE	Res.	Res.	*SSOE*	TXDMAEN	RXDMAEN
								rw	rw	rw			rw	rw	rw

Бит 7. TXEIE(Tx buffer empty interrupt enable) — единица в этом бите разрешает прерывание, когда буфер передачи пуст.

Бит 6. RXNEIE(RX buffer not empty interrupt enable) — единица в этом бите разрешает прерывание когда буфер не пуст.

Бит 5. ERRIE(Error interrupt enable) — единица в этом бите разрешает прерывание при возникновении ошибки, о которой сигнализируют биты CRCERR, OVR, MODF.

Бит 2. SSOE(SS output enable) — единица в этом бите, разрешает использовать вывод NSS в качестве выхода и модуль SPI сам управляет выводом NSS, но управляет ним по следующему алгоритму: при включении модуля SPI(SPE = 1) на NSS появляется низкий уровень и сохраняется пока модуль не будет выключен(SPE = 0), что в большинстве случаев не подходит, поэтому для этих целей использую GPIO, а на NSS в режиме Master программно выставляю единицу(SSM = 1, SSI = 1).

Бит 1. TXDMAEN(Tx buffer DMA enable) — разрешает/запрещает формировать запрос к DMA при появлении флага TXE(по завершении передачи).

Бит 0. RXDMAEN(Rx buffer DMA enable) — разрешает/запрещает формировать запрос к DMA при появлении флага RXE(по завершении приёма).

SPI_SR

Статусный регистр **SPI_SR**.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								BSY	OVR	MODF	CRC ERR	UDR	CHSIDE	TXE	RXNE
								r	r	r	rc_w0	r	r	r	r

Бит 7. BSY(Busy flag) — флаг занятости, предназначен для отслеживания состояния сдвигового регистра, устанавливается аппаратно во время обмена данными, а точнее с момента когда данные из регистра SPI_DR поступают в сдвиговый регистр, сбрасывается при опустошении сдвигового регистра, то есть когда все данные ушли в линию MOSI.

Производители не советуют проверять его при приёме и передаче, вместо этого советуют пользоваться флагами TXE и RXNE.

Бит 6. OVR(Overrun flag) – устанавливается при переполнении буфера, когда новые данные перезаписали старые, которые ещё не были прочитаны.

Бит 5. MODF(Mode fault) — устанавливается если в режиме Master на вход NSS поступает сигнал низкого уровня, и он становится Slave. Сбросить его можно с помощью специальной последовательности.

Бит 4. CRCERR(CRC error flag) – флаг ошибки контрольной суммы.

Бит 1.TXE(Transmit buffer empty) – устанавливается когда буфер передачи(регистр SPI_DR) пуст, очищается при загрузке данных

Бит 0. RXNE(Receive buffer not empty) — устанавливается когда приёмный буфер содержит данные, очищается при считывании данных.

SPI_DR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Регистр данных, фактически он состоит из двух буферов: приёма и передачи, но мы работаем только с регистром DR, поэтому для отправки данных пишем в регистр DR, для приёма читаем его же.

Функции HAL для работы с SPI

HAL_SPI_Init

*HAL_StatusTypeDef HAL_SPI_Init(SPI_HandleTypeDef *hspi)*

Инициализирует работу SPI.

Параметры:

hspi – указатель на структуру SPI_HandleTypeDef

Возвращаемое значение: *HAL_StatusTypeDef – HAL_OK.*

Структура SPI_HandleTypeDef

```
typedef struct __SPI_HandleTypeDef
{
  SPI_TypeDef                    *Instance;
  SPI_InitTypeDef                Init;
  uint8_t                             *pTxBuffPtr;
  uint16_t                            TxXferSize;
  __IO uint16_t                  TxXferCount;
  uint8_t                             *pRxBuffPtr;
  uint16_t                            RxXferSize;
  __IO uint16_t                  RxXferCount;
  void (*RxISR)(struct __SPI_HandleTypeDef *hspi);
  void (*TxISR)(struct __SPI_HandleTypeDef *hspi);
  DMA_HandleTypeDef              *hdmatx;
  DMA_HandleTypeDef              *hdmarx;
  HAL_LockTypeDef                Lock;
  __IO HAL_SPI_StateTypeDef      State;
  __IO uint32_t                  ErrorCode;
};
```

1. *Instance*:
   - Тип: SPI_TypeDef *
   - Описание: Указатель на регистры периферийного модуля SPI (например, SPI1, SPI2, SPI3 и т.д.).
   - Пример: SPI1 или SPI2.
2. *Init*:
   - Тип: SPI_InitTypeDef
   - Описание: Структура, содержащая настройки SPI. Включает следующие поля:
     - Mode: Режим работы SPI (SPI_MODE_MASTER или SPI_MODE_SLAVE).

    - Direction: Направление передачи данных (SPI_DIRECTION_2LINES, SPI_DIRECTION_2LINES_RXONLY, SPI_DIRECTION_1LINE).
    - DataSize: Размер данных (SPI_DATASIZE_8BIT, SPI_DATASIZE_16BIT).
    - CLKPolarity: Полярность тактового сигнала (SPI_POLARITY_LOW, SPI_POLARITY_HIGH).
    - CLKPhase: Фаза тактового сигнала (SPI_PHASE_1EDGE, SPI_PHASE_2EDGE).
    - NSS: Управление сигналом NSS (Slave Select) (SPI_NSS_SOFT, SPI_NSS_HARD_INPUT, SPI_NSS_HARD_OUTPUT).
    - BaudRatePrescaler: Предделитель частоты (SPI_BAUDRATEPRESCALER_2, SPI_BAUDRATEPRESCALER_4, ..., SPI_BAUDRATEPRESCALER_256).
    - FirstBit: Порядок передачи битов (SPI_FIRSTBIT_MSB, SPI_FIRSTBIT_LSB).
    - TIMode: Режим TI (Texas Instruments) (SPI_TIMODE_DISABLE, SPI_TIMODE_ENABLE).
    - CRCCalculation: Включение расчета CRC (SPI_CRCCALCULATION_DISABLE, SPI_CRCCALCULATION_ENABLE).
    - CRCPolynomial: Полином для расчета CRC (если включен).
3. *pTxBuffPtr*:
    - Тип: uint8_t * или uint16_t *
    - Описание: Указатель на буфер передаваемых данных.
4. *TxXferSize*:
    - Тип: uint16_t
    - Описание: Размер данных для передачи (в байтах или словах, в зависимости от DataSize).
5. *TxXferCount*:
    - Тип: uint16_t
    - Описание: Счетчик оставшихся данных для передачи.
6. *pRxBuffPtr*:
    - Тип: uint8_t * или uint16_t *
    - Описание: Указатель на буфер принимаемых данных.
7. *RxXferSize*:
    - Тип: uint16_t
    - Описание: Размер данных для приема (в байтах или словах, в зависимости от DataSize).
8. *RxXferCount*:
    - Тип: uint16_t
    - Описание: Счетчик оставшихся данных для приема.
9. *hdmatx* и *hdmarx*:

- Тип: DMA_HandleTypeDef *
- Описание: Указатели на структуры DMA для передачи и приема данных, если используется DMA.

10. *Lock*:
    - Тип: __IO HAL_LockTypeDef
    - Описание: Блокировка для защиты от одновременного доступа к ресурсам SPI.
11. *State*:
    - Тип: HAL_SPI_StateTypeDef
    - Описание: Текущее состояние SPI. Возможные значения:
        - HAL_SPI_STATE_RESET: SPI не инициализирован.
        - HAL_SPI_STATE_READY: SPI готов к работе.
        - HAL_SPI_STATE_BUSY: SPI занят передачей или приемом данных.
        - HAL_SPI_STATE_BUSY_TX: SPI занят передачей данных.
        - HAL_SPI_STATE_BUSY_RX: SPI занят приемом данных.
        - HAL_SPI_STATE_BUSY_TX_RX: SPI занят одновременной передачей и приемом данных.
        - HAL_SPI_STATE_ERROR: Ошибка в работе SPI.
12. *ErrorCode*:
    - Тип: uint32_t
    - Описание: Код ошибки. Может содержать комбинацию следующих флагов:
        - HAL_SPI_ERROR_NONE: Ошибок нет.
        - HAL_SPI_ERROR_MODF: Ошибка режима (Mode Fault).
        - HAL_SPI_ERROR_CRC: Ошибка CRC.
        - HAL_SPI_ERROR_OVR: Ошибка переполнения (Overrun).
        - HAL_SPI_ERROR_FRE: Ошибка формата (Frame Error).
        - HAL_SPI_ERROR_DMA: Ошибка DMA.
13. *RxISR* и *TxISR*:
    - Тип: Указатели на функции.
    - Описание: Указатели на функции обработки прерываний для приема и передачи данных.

HAL_SPI_Transmit

*HAL_StatusTypeDef HAL_SPI_Transmit(SPI_HandleTypeDef *hspi, uint8_t *pData, uint16_t Size, uint32_t Timeout);*

Передает данные по протоколу SPI.

Параметры:

hspi – указатель на структуру SPI_HandleTypeDef

pData – указатель на буфер данных;

Size – размер буфера данных (в байтах)

Timeout – таймаут на выполнение функции HAL.

Возвращаемое значение: *HAL_StatusTypeDef – HAL_OK, HAL_TIMEOUT, HAL_BUSY.*

HAL_SPI_Receive

```
HAL_StatusTypeDef HAL_SPI_Receive(SPI_HandleTypeDef *hspi, uint8_t *pData, uint16_t Size, uint32_t Timeout);
```

Принимает данные по протоколу SPI.

Параметры:

hspi – указатель на структуру SPI_HandleTypeDef

pData – указатель на буфер, куда нужно записать данные;

Size – размер буфера данных (в байтах)

Timeout – таймаут на выполнение функции HAL.

Возвращаемое значение: *HAL_StatusTypeDef – HAL_OK, HAL_TIMEOUT, HAL_BUSY.*

HAL_SPI_TransmitReceive

```
HAL_StatusTypeDef HAL_SPI_TransmitReceive(SPI_HandleTypeDef *hspi, uint8_t *pTxData, uint8_t *pRxData, uint16_t Size, uint32_t Timeout);
```

Передает и принимает данные по протоколу SPI.

Параметры:

- hspi – указатель на структуру SPI_HandleTypeDef
- pTxData – указатель на буфер передаваемых данных;
- pRxData – указатель на буфер, куда нужно записать данные;
- Size – размер буфера данных (в байтах). Распространяется и на tx, и на rx;
- Timeout – таймаут на выполнение функции HAL.

Возвращаемое значение: *HAL_StatusTypeDef – HAL_OK, HAL_TIMEOUT, HAL_BUSY.*

Функции HAL SPI для выполнения в режиме прерывания

```
HAL_StatusTypeDef    HAL_SPI_Transmit_IT(SPI_HandleTypeDef    *hspi, uint8_t *pData, uint16_t Size);
```

```
HAL_StatusTypeDef HAL_SPI_Receive_IT(SPI_HandleTypeDef *hspi, uint8_t *pData, uint16_t Size);
```

```
HAL_StatusTypeDef       HAL_SPI_TransmitReceive_IT(SPI_HandleTypeDef *hspi, uint8_t *pTxData, uint8_t *pRxData, uint16_t Size);
```

Функции обратного вызова SPI для обработки данных в режиме прерываний

Глобальный обработчик (автоматически генерируется)

```
void HAL_SPI_IRQHandler(SPI_HandleTypeDef *hspi);
```

Обработчики на конец приема и/или передачи (нужно прописывать вручную)

```
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi);
```

```
void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef *hspi);
```

```
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi);
```

Обработчики на половину процесса приема и/или передачи (нужно прописывать вручную)

```
void HAL_SPI_TxHalfCpltCallback(SPI_HandleTypeDef *hspi);
```

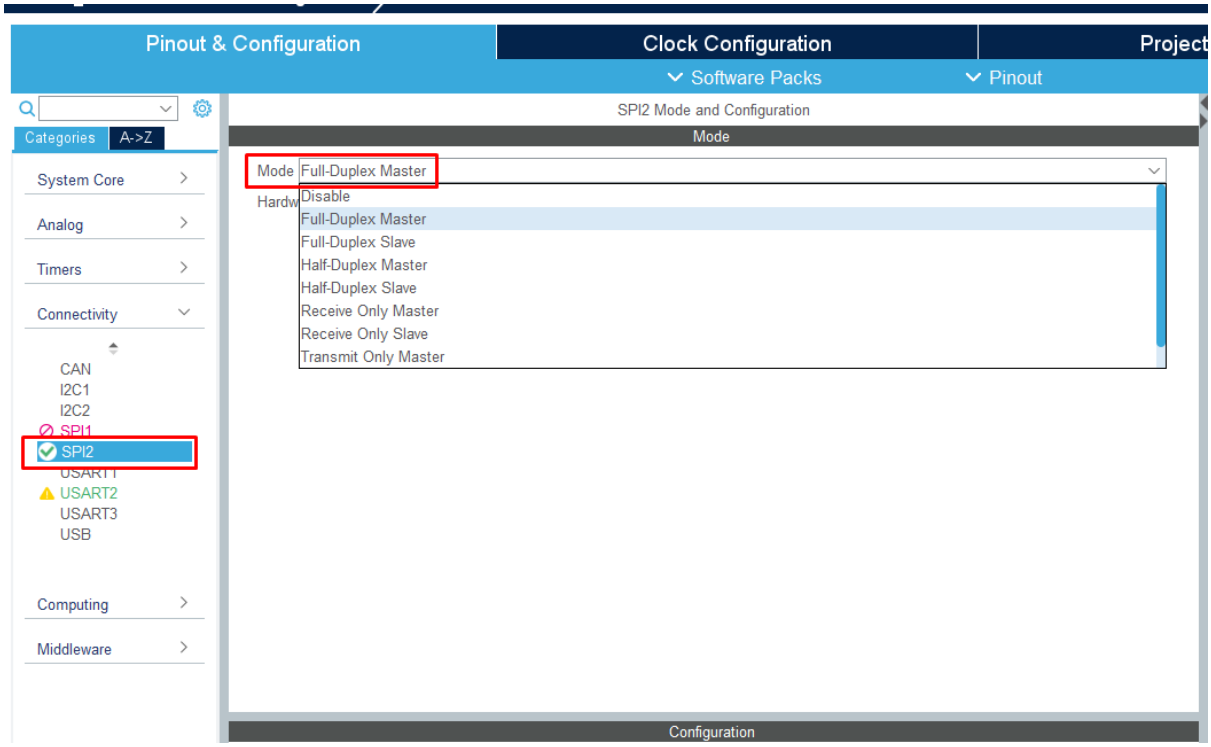
```
void HAL_SPI_RxHalfCpltCallback(SPI_HandleTypeDef *hspi);
```

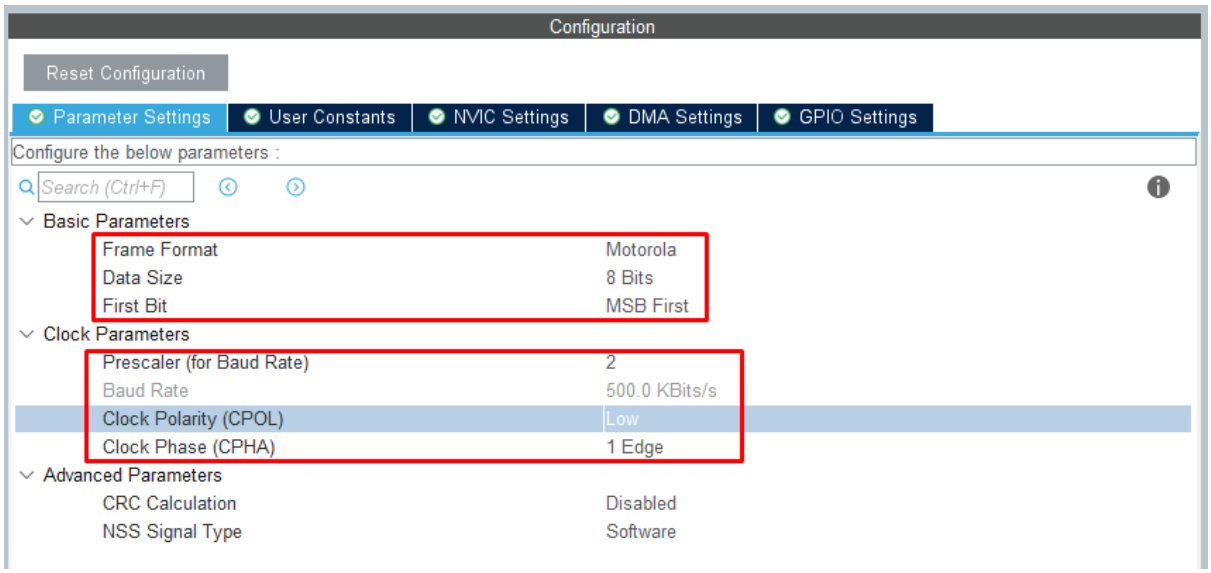
```
void HAL_SPI_TxRxHalfCpltCallback(SPI_HandleTypeDef *hspi);
```

Обработчик ошибки работы SPI

void HAL_SPI_ErrorCallback(SPI_HandleTypeDef *hspi);

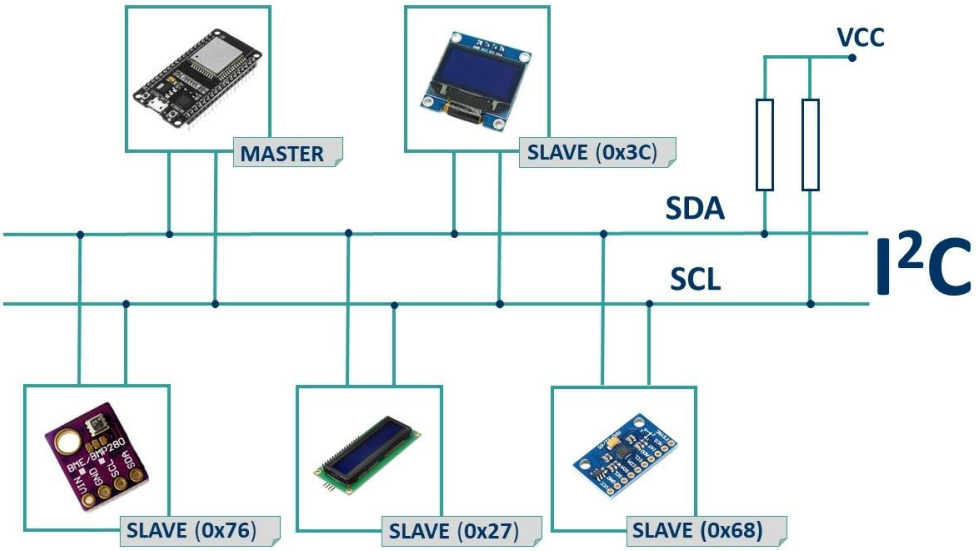
Ниже представлены скриншоты инициализации периферии SPI, и настройка его параметров.


Pinout & Configuration
Clock Configuration
Project
Software Packs
Pinout
Categories
A->Z
System Core
Analog
Timers
Connectivity
CAN
I2C1
I2C2
SPI1
SPI2
USART1
USART2
USART3
USB
Computing
Middleware
SPI2 Mode and Configuration
Mode
Mode
Full-Duplex Master
Disable
Full-Duplex Master
Full-Duplex Slave
Half-Duplex Master
Half-Duplex Slave
Receive Only Master
Receive Only Slave
Transmit Only Master
Configuration


Configuration
Reset Configuration
Parameter Settings
User Constants
NVIC Settings
DMA Settings
GPIO Settings
Configure the below parameters :
Search (Ctrl+F)
Basic Parameters
Frame Format
Motorola
Data Size
8 Bits
First Bit
MSB First
Clock Parameters
Prescaler (for Baud Rate)
2
Baud Rate
500.0 KBits/s
Clock Polarity (CPOL)
Low
Clock Phase (CPHA)
1 Edge
Advanced Parameters
CRC Calculation
Disabled
NSS Signal Type
Software

Протокол I2C (Inter-Integrated Circuit)

I2C (Inter-Integrated Circuit) — это последовательный протокол связи, разработанный компанией Philips (ныне NXP Semiconductors). Он используется для обмена данными между микроконтроллерами и периферийными устройствами, такими как датчики, EEPROM, дисплеи и другие. I2C популярен благодаря своей простоте, использованию всего двух линий для связи и поддержке множества устройств на одной шине.



Основные характеристики I2C:

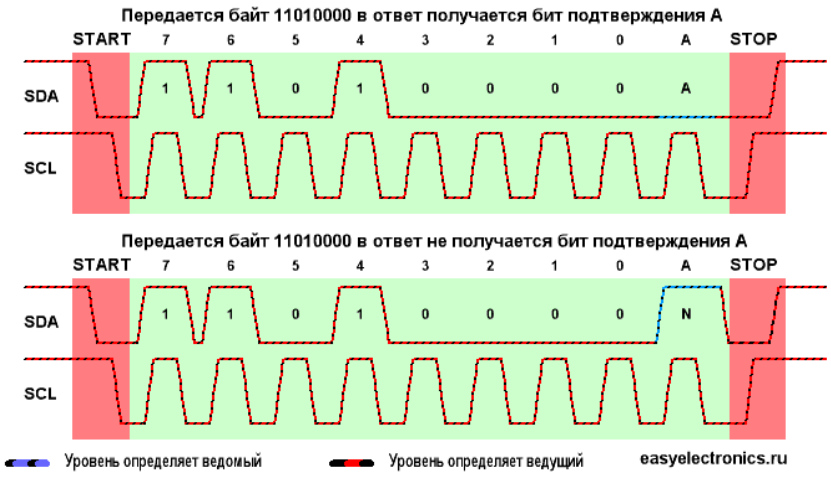
1. **Двухпроводная шина**: Используются две линии:
   - **SDA (Serial Data Line)**: Линия данных.
   - **SCL (Serial Clock Line)**: Линия тактового сигнала.
2. **Мультимастерность**: На одной шине может быть несколько мастеров.
3. **Адресация устройств**: Каждое устройство на шине имеет уникальный 7-битный или 10-битный адрес.
4. **Низкая скорость**: По сравнению с SPI, I2C работает на более низких скоростях (обычно до 400 кбит/с в режиме Fast Mode).
5. **Поддержка множества устройств**: На одной шине может быть до 128 устройств (для 7-битной адресации).

Основные принципы работы I2C:

1. **Мастер и слейвы**:
   - Мастер инициирует передачу данных и генерирует тактовый сигнал (SCL).
   - Слейвы отвечают на запросы мастера.
2. **Адресация**:
   - Мастер отправляет адрес слейва перед началом передачи данных.
   - Адрес состоит из 7 или 10 бит, за которыми следует бит чтения/записи (R/W).

3. **Форматы сообщений**:
    - **Стартовый бит (Start)**: Мастер инициирует передачу, опуская SDA при высоком уровне SCL.
    - **Стоповый бит (Stop)**: Мастер завершает передачу, поднимая SDA при высоком уровне SCL.
    - **Повторный старт (Repeated Start)**: Мастер может начать новую передачу без завершения предыдущей.
4. **Подтверждение (ACK/NACK)**:
    - После каждого байта данных получатель отправляет бит подтверждения (ACK) или отрицания (NACK).

Наглядная демонстрация принципа работы I2C представлена на рисунке ниже:



Формат передачи данных:

1. Мастер отправляет стартовый бит.
2. Мастер отправляет адрес слейва (7 или 10 бит) и бит R/W (0 для записи, 1 для чтения).
3. Слейв отправляет ACK.
4. Мастер или слейв передают данные (по 8 бит за раз).
5. После каждого байта получатель отправляет ACK или NACK.
6. Мастер отправляет стоповый бит для завершения передачи.

Режимы работы I2C:

1. **Стандартный режим (Standard Mode)**: До 100 кбит/с.
2. **Быстрый режим (Fast Mode)**: До 400 кбит/с.
3. **Высокоскоростной режим (High-Speed Mode)**: До 3,4 Мбит/с. Не поддерживается в STM32F103!

Регистр управления I2C_CR1

I2C_CR1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWRST	Res.	ALERT	PEC	POS	ACK	STOP	START	NO STRETCH	ENGC	ENPEC	ENARP	SMB TYPE	Res.	SMBUS	PE
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw

Бит 15. SWRST(Software reset) — единица в этом бите сбрасывает значение всех регистров модуля в дефолтное стояние, может использоваться для сброса при возникновении ошибки.

Бит 13. ALERT(SMBus alert) — установка единицы в этот бит разрешает генерировать сигнал alert в режиме SMBus.

Бит 12. PEC(Packet error checking) — управление этим битом производится программно, но он может быть сброшен аппаратно когда передается PEC, START, STOP или PE=0. Единица в этом бите разрешает передачу CRC.

Бит 11. POS(Acknowledge/PEC Position (for data reception)) — состояние этого бита определяет положение ACK/PEC в двух байтовой конфигурации в режиме Master.

Бит 10. ACK(Acknowledge enable) — единица в этом бите разрешает отправлять ACK/NACK после приема байта адреса или данных.

Бит 9. STOP(Stop generation) — установка единицы в этот бит генерирует сигнал STOP в режиме Master.

Бит 8. START(Start generation) — установка единицы в этот бит генерирует состояние START в режиме Master,

Бит 7. NOSTRETCH(Clock stretching disable (Slave mode)) — если на обработку данных требуется время Slave может остановить передачу мастера, прижав линию SCL к земле, Master будет ждать и не будет ни чего слать, пока линия не будет отпущена. Ноль в этом бите прижимает SCL к земле.

Бит 6. ENGC(General call enable) — если в этом бите установлена единица, модуль отвечает ACKом на широковещательный адрес 0x00.

Бит 5. ENPEC(PEC enable) — установка единицы в этот бит включает аппаратный подсчет CRC.

Бит 4. ENARP(ARP enable) — установка единицы в этот бит включает ARP.

Бит 3. SMBTYPE(SMBus type) — если в этом бите установлен ноль модуль работает в режиме Slave, если единица в режиме Master.

Бит 1. SMBUS(SMBus mode) — если в этом бите установлен ноль модуль работает в режиме I2C, если единица SMBus.

Бит 0. PE(Peripheral enable) — единица в этом бите включает модуль.

I2C_CR2

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			LAST	DMAEN	ITBUFEN	ITEVTEN	ITERREN	Reserved		FREQ[5:0]					
			rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

Бит 12. LAST(DMA last transfer) — единица в этом бите разрешает DMA генерировать сигнал окончания передачи EOT(End of Transfer).

Бит 11. DMAEN(DMA requests enable) — единица в этом бите разрешает делать запрос к DMA при установке флагов TxE или RxNE.

Бит 10. ITBUFEN(Buffer interrupt enable) — если этот бит сброшен, разрешены все прерывания, кроме прерываний по приему и передаче.

Бит 9. ITEVTEN(Event interrupt enable) — единица в этом бите разрешает прерывания по событию.

Бит 8. ITERREN(Error interrupt enable) — единица в этом бите разрешает прерывания при возникновении ошибок.

Бит 5-0. FREQ[5:0](Peripheral clock frequency) — в это битовое битовое поле необходимо записать частоту тактирования модуля, она может принимать значение от 2 до 50.

I2C_OAR1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD MODE	Reserved					ADD[9:8]		ADD[7:1]							ADD0
rw						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Бит 15. ADDMODE(Addressing mode) — этот бит определяет размер адреса Slave, ноль соответствует размеру адреса 7 бит, единица — 10 бит.

Бит 9-8. ADD[9:8](Interface address) — старшие биты адреса, в случае если адрес 10-битный.

Бит 7-1. ADD[1:7](Interface address) — адрес устройства.

Бит 0. ADD0(Interface address) — младший бит адреса, в случае если адрес 10-битный..

I2C_OAR2

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								ADD2[7:1]							ENDUAL
								rw	rw	rw	rw	rw	rw	rw	rw

Бит 7-1. ADD2[7:1] — альтернативный адрес на который будет отзываться Slave.

Бит 0. ENDUAL(Dual addressing mode enable) — единица в этом бите разрешает Slave отзываться на альтернативный адрес в 7-битном режиме.

I2C_DR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								DR[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw

I2C_DR — регистр данных, для отправки данных пишем в регистр DR, для приёма читаем его же.

I2C_SR1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMB ALERT	TIME OUT	Res.	PEC ERR	OVR	AF	ARLO	BERR	TxE	RxNE	Res.	STOPF	ADD10	BTF	ADDR	SB
rc_w0	rc_w0		rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	r	r		r	r	r	r	r

Бит 15. SMBALERT(SMBus alert) — возникает в случае alert в шине SMBus.

Бит 14. TIMEOUT (Timeout or Tlow error) — возникает если линия SCL прижата к земле. Для master 10mS, для slave 25mS.

Бит 12. PECERR (PEC Error in reception) — возникает при ошибке PEC при приеме.

Бит 11. OVR (Overrun/Underrun) — возникает при переполнении данных.

Бит 10. AF (Acknowledge failure) — устанавливается при получении сигнала NACK. Для сброса нужно записать 0.

Бит 9. ARLO (Arbitration lost (master mode)) — устанавливается при потере арбитража. Для сброса нужно записать 0.

Бит 8. BERR (Bus error) — ошибка шины. Устанавливается в случае возникновения сигнала START или STOP в неправильный момент.

Бит 7. TxE (Data register empty (transmitters)) — устанавливается при опустошении регистра DR, а точнее когда данные из него были перемещены в сдвиговый регистр.

Бит 6. RxNE (Data register not empty (receivers)) — устанавливается при приеме байта данных, кроме адреса.

Бит 4. STOPF(Stop detection (slave mode)) — при работе в режиме slave устанавливается при обнаружении сигнала STOP, если перед этим был сигнал ACK. Для сброса необходимо прочитать SR1 и произвести запись в CR1.

Бит 3. ADD10 (10-bit header sent (Master mode)) — устанавливается при отправке первого байта 10-битного адреса.

Бит 2. BTF (Byte transfer finished) — флаг устанавливается по окончании приема/передачи байта, работает только при NOSTRETCH равном нулю.

Бит 1. ADDR(Address sent (master mode)/matched (slave mode)) — в режиме master устанавливается после передачи адреса, в режиме slave устанавливается при совпадении адреса. Для сброса нужно прочитать регистр SR1, а затем SR2.

Бит 0. SB(Start bit (Master mode)) — устанавливается при возникновении сигнала START. Для сброса флага необходимо прочитать SR1 и записать данные в регистр DR .

I2C_SR2

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PEC[7:0]								DUALF	SMB HOST	SMBDE FAULT	GEN CALL	Res.	TRA	BUSY	MSL
r	r	r	r	r	r	r	r	r	r	r	r		r	r	r

Бит 15-8. PEC[7:0](Packet error checking register) — в это битовое поле записывается контрольная сумма кадра.

Бит 7. DUALF(Dual flag (Slave mode)) — ноль в этом бите говорит о том, что адрес который принял Slave соответствует OAR1, иначе OAR2.

Бит 6. SMBHOST(SMBus host header (Slave mode)) — устанавливается, когда принят заголовок SMBus Host.

Бит 5.SMBDEFAULT(SMBus device default address (Slave mode)) — устанавливается, если принят адрес по умолчанию для SMBus-устройства.

Бит 4. GENCALL(General call address (Slave mode)) — устанавливается, если принят широковещательный адрес в режиме ведомого.

Бит 2. TRA(Transmitter/receiver) — единица в этом бите говорит о том, что модуль работает как передатчик, иначе приемник.

Бит 1. BUSY(Bus busy) — флаг занятости.

Бит 0. MSL(Master/slave) — единица в этом бите говорит о том, что модуль работает в режиме Master, иначе Slave.

I2C_CCR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
F/S	DUTY	Reserved		CCR[11:0]											
rw	rw			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Бит 15. F/S(I2C master mode selection) — при установке единицы в этот бит модуль работает в режиме FAST, иначе STANDART.

Бит 14. DUTY (Fm mode duty cycle) — этот бит задает скважность сигнала SCL в режиме FAST. Если установлен ноль, tlow/thigh = 2, иначе tlow/thigh = 16/9.

Бит 11-0. CCR[11:0](Clock control register in Fm/Sm mode (Master mode)) — при работе в режиме Master задает тактовую частоту линии SCL.

Функции HAL для работы с I2C

HAL_I2C_Init

```
HAL_StatusTypeDef HAL_I2C_Init(I2C_HandleTypeDef *hi2c);
```

Инициализация периферии I2C

Параметр hi2c – указатель на структуру I2C_HandleTypeDef.

Возвращает *HAL_StatusTypeDef (HAL_OK).*

Структура I2C_HandleTypeDef

```
typedef struct
{
      I2C_TypeDef                *Instance;   // Указатель на базовый адрес I2C
      I2C_InitTypeDef            Init;        // Структура инициализации I2C
      uint8_t                    *pBuffPtr;   // Указатель на буфер данных
      uint16_t                   XferSize;    // Размер передачи
      __IO uint16_t              XferCount;   // Счетчик переданных данных
      uint32_t                   ErrorCode;   // Код ошибки
      HAL_LockTypeDef            Lock;   // Блокировка для многопоточной среды
      HAL_I2C_StateTypeDef       State;  // Состояние I2C
        __IO uint32_t          Devaddress;   /* Целевой адрес устройства */
        __IO uint32_t          Memaddress; /* Адрес памяти буфера данных*/
        __IO uint32_t          MemaddSize; /* Размер буфера*/
        __IO uint32_t           EventCount; /* Количество произошедших событий I2C
      */
} I2C_HandleTypeDef;
```

Instance:

Указатель на базовый адрес I2C (например, I2C1, I2C2 и т.д.), который используется для доступа к регистрам I2C.

Init:

Структура I2C_InitTypeDef, содержащая параметры конфигурации I2C, такие как:

- ClockSpeed: Скорость тактирования I2C.
- DutyCycle: Доля такта для режима Fast Mode.
- OwnAddress1: Собственный адрес устройства I2C.
- AddressingMode: Режим адресации (7-битный или 10-битный).
- DualAddressMode: Режим двойного адреса.
- OwnAddress2: Вторичный адрес устройства.
- GeneralCallMode: Режим общего вызова.
- NoStretchMode: Режим без растяжения такта.

pBuffPtr:

Указатель на буфер данных, который используется для передачи или приема данных по I2C.

XferSize:

Размер передачи в байтах. Это количество байтов, которое необходимо передать или получить.

XferCount:

Счетчик переданных или принятых байтов. Используется для отслеживания прогресса передачи.

ErrorCode:

Код ошибки, который может возникнуть во время работы с I2C. Это поле позволяет отслеживать и обрабатывать ошибки, такие как переполнение, тайм-аут и т.д.

Lock:

Структура блокировки для многопоточной среды, которая предотвращает одновременный доступ к периферии I2C из разных потоков.

State:

Переменная, указывающая текущее состояние I2C, например:

- HAL_I2C_STATE_RESET: Инициализация не выполнена.
- HAL_I2C_STATE_READY: I2C готов к работе.
- HAL_I2C_STATE_BUSY: I2C занят передачей данных.
- HAL_I2C_STATE_BUSY_TX: I2C занят передачей данных.
- HAL_I2C_STATE_BUSY_RX: I2C занят приемом данных.
- HAL_I2C_STATE_ERROR: Произошла ошибка.

HAL_I2C_Master_Transmit

```
HAL_StatusTypeDef  HAL_I2C_Master_Transmit(I2C_HandleTypeDef  *hi2c, uint16_t DevAddress, uint8_t *pData, uint16_t Size, uint32_t Timeout);
```

Передает данные в режиме слейва.

Параметры:

- hi2c - указатель на структуру I2C;
- DevAddress – адрес устройства, на которое передаются данные;
- pData – указатель на буфер с передаваемыми данными;
- Size – размер буфера передаваемых данных;
- Timeout – таймаут на выполнение функции

Возвращает *HAL_StatusTypeDef: HAL_OK, HAL_BUSY, HAL_ERROR, HAL_TIMEOUT.*

HAL_I2C_Master_Receive

```
HAL_StatusTypeDef  HAL_I2C_Master_Receive(I2C_HandleTypeDef  *hi2c, uint16_t DevAddress, uint8_t *pData, uint16_t Size, uint32_t Timeout);
```

Принимает данные в режиме слейва.

Параметры:

- hi2c - указатель на структуру I2C;
- DevAddress – адрес устройства, от которого принимаются данные;
- pData – указатель на буфер, в который запишутся принятые данные данными;
- Size – размер буфера;
- Timeout – таймаут на выполнение функции

Возвращает *HAL_StatusTypeDef: HAL_OK, HAL_BUSY, HAL_ERROR, HAL_TIMEOUT.*

HAL_I2C_Slave_Transmit

```
HAL_StatusTypeDef  HAL_I2C_Slave_Transmit(I2C_HandleTypeDef  *hi2c,
uint8_t *pData, uint16_t Size, uint32_t Timeout);
```

Передает данные в режиме мастера.

Параметры:

- hi2c - указатель на структуру I2C;
- DevAddress – адрес устройства, на которое передаются данные;
- pData – указатель на буфер с передаваемыми данными;
- Size – размер буфера передаваемых данных;
- Timeout – таймаут на выполнение функции

Возвращает *HAL_StatusTypeDef: HAL_OK, HAL_BUSY, HAL_ERROR, HAL_TIMEOUT.*

HAL_I2C_Slave_Receive

```
HAL_StatusTypeDef  HAL_I2C_Slave_Receive(I2C_HandleTypeDef  *hi2c,
uint8_t *pData, uint16_t Size, uint32_t Timeout);
```

Принимает данные в режиме мастера.

Параметры:

- hi2c - указатель на структуру I2C;
- DevAddress – адрес устройства, от которого принимаются данные;
- pData – указатель на буфер, в который запишутся принятые данные данными;
- Size – размер буфера;
- Timeout – таймаут на выполнение функции

Возвращает *HAL_StatusTypeDef: HAL_OK, HAL_BUSY, HAL_ERROR, HAL_TIMEOUT.*

HAL_I2C_Mem_Write

```
HAL_StatusTypeDef HAL_I2C_Mem_Write(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size, uint32_t Timeout);
```

Передает данные на запись в конкретный участок памяти слейва (по адресу памяти).

Параметры:

- hi2c - указатель на структуру I2C;
- DevAddress – адрес устройства, куда передаются данные;
- MemAddress– адрес памяти слейва, куда записываются данные;
- MemAddSize– размер памяти начиная с адреса, куда пишутся данные;
- pData – указатель на буфер с данными;
- Size – размер буфера;
- Timeout – таймаут на выполнение функции

Возвращает *HAL_StatusTypeDef: HAL_OK, HAL_BUSY, HAL_ERROR, HAL_TIMEOUT.*

HAL_I2C_Mem_Read

```
HAL_StatusTypeDef HAL_I2C_Mem_Read(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size, uint32_t Timeout);
```

Принимает данные из конкретного участка памяти слейва (с адреса памяти).

Параметры:

- hi2c - указатель на структуру I2C;
- DevAddress – адрес устройства, откуда передаются данные;
- MemAddress– адрес памяти слейва, откуда читаются данные;
- MemAddSize– размер памяти начиная с адреса, откуда читаются данные;
- pData – указатель на буфер, в который запишутся принятые данные данными;
- Size – размер буфера;
- Timeout – таймаут на выполнение функции

Возвращает *HAL_StatusTypeDef: HAL_OK, HAL_BUSY, HAL_ERROR, HAL_TIMEOUT.*

HAL_I2C_IsDeviceReady

```
HAL_StatusTypeDef HAL_I2C_IsDeviceReady(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint32_t Trials, uint32_t Timeout);
```

Получает статус готовности устройства

Параметры:

- hi2c - указатель на структуру I2C;
- DevAddress – адрес устройства, статус которого нужно получить;
- Trials– сколько нужно сделать попыток чтения статуса;
- Timeout – таймаут на выполнение функции

Возвращает *HAL_StatusTypeDef: HAL_OK, HAL_BUSY, HAL_ERROR, HAL_TIMEOUT.*

Функции при работе в режиме прерываний

```
HAL_StatusTypeDef HAL_I2C_Master_Transmit_IT(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint8_t *pData, uint16_t Size);
```

```
HAL_StatusTypeDef HAL_I2C_Master_Receive_IT(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint8_t *pData, uint16_t Size);
```

```
HAL_StatusTypeDef HAL_I2C_Slave_Transmit_IT(I2C_HandleTypeDef *hi2c, uint8_t *pData, uint16_t Size);
```

```
HAL_StatusTypeDef HAL_I2C_Slave_Receive_IT(I2C_HandleTypeDef *hi2c, uint8_t *pData, uint16_t Size);
```

```
HAL_StatusTypeDef HAL_I2C_Mem_Write_IT(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size);
```

```
HAL_StatusTypeDef HAL_I2C_Mem_Read_IT(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size);
```

Обработчики прерываний

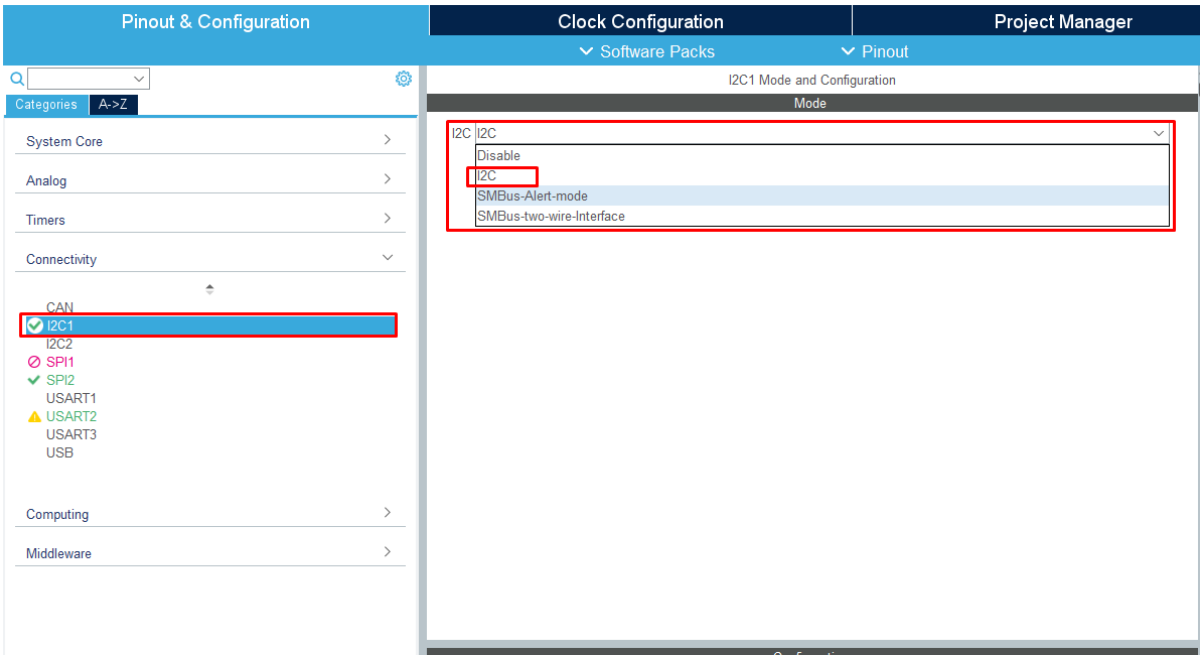
Глобальные обработчики (автоматически генерируются и вызываются)

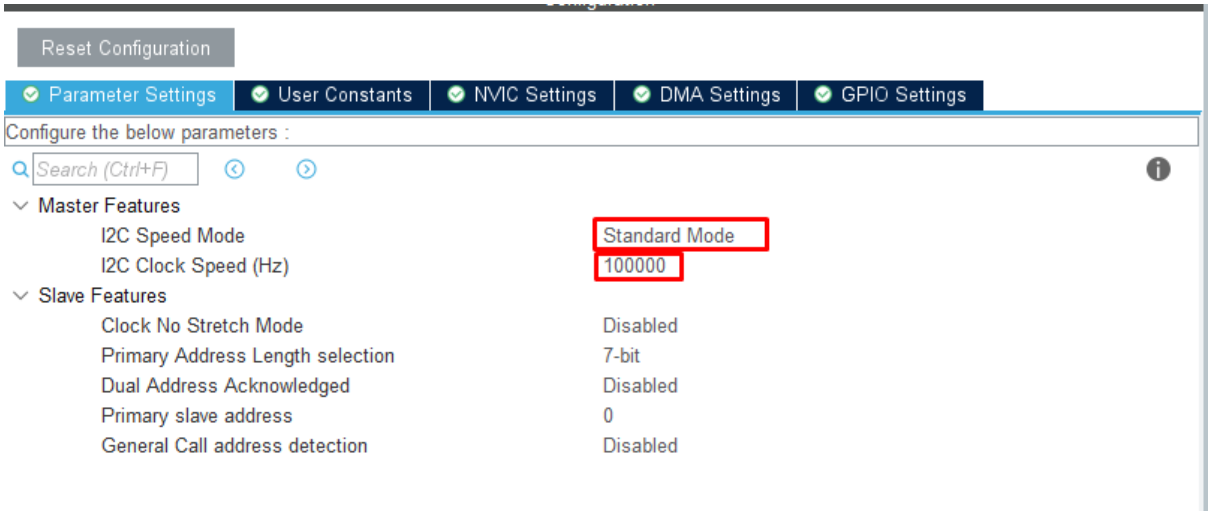
```
void HAL_I2C_EV_IRQHandler(I2C_HandleTypeDef *hi2c);

void HAL_I2C_ER_IRQHandler(I2C_HandleTypeDef *hi2c);
```

Функции обратного вызова (прописываются вручную)

```
void HAL_I2C_MasterTxCpltCallback(I2C_HandleTypeDef *hi2c);

void HAL_I2C_MasterRxCpltCallback(I2C_HandleTypeDef *hi2c);

void HAL_I2C_SlaveTxCpltCallback(I2C_HandleTypeDef *hi2c);

void HAL_I2C_SlaveRxCpltCallback(I2C_HandleTypeDef *hi2c);

void HAL_I2C_MemTxCpltCallback(I2C_HandleTypeDef *hi2c);

void HAL_I2C_MemRxCpltCallback(I2C_HandleTypeDef *hi2c);

void HAL_I2C_ErrorCallback(I2C_HandleTypeDef *hi2c);
```

Ниже представлены скриншоты базовой настройки I2C в STM32CubeIDE




Reset Configuration
Parameter Settings
User Constants
NVIC Settings
DMA Settings
GPIO Settings
Configure the below parameters :
Search (Ctrl+F)
Master Features
I2C Speed Mode
Standard Mode
I2C Clock Speed (Hz)
100000
Slave Features
Clock No Stretch Mode
Disabled
Primary Address Length selection
7-bit
Dual Address Acknowledged
Disabled
Primary slave address
0
General Call address detection
Disabled

Задание

1. Настроить на двух платах с микроконтроллерами STM32 следующие режимы работы:
   - **SPI-master full-duplex;**
   - **SPI-slave full-duplex.**

Учтите, что **параметры скоростей и уровней сигналов CPOL и CPHA должны совпадать!**

2. Реализуйте передачу строки “Blink” (или любую другую комбинацию символов на ваше усмотрение) по SPI с вашего STM-мастера на STM-slave.

3. На вашем STM-slave реализуйте приём данной строки по SPI.

4. Реализуйте программу на STM-slave, которая будет мигать светодиодом в случае, если пришла нужная строка.

5. Реализуйте на STM-slave передачу строки “Blink has finished” по SPI, которая будет передаваться после завершения мигания светодиодом, в STM-мастер.

6. На вашем STM-master реализуйте приём данной строки по SPI.

7. Реализуйте пункты заданий с **1** по **7**, но теперь для протокола **I2C**.

8. Если отсутствует вторая плата STM32, но имеется внешняя плата, работающая по SPI или I2C, то ***вместо заданий с 1 по 7*** реализуйте работу с внешней платой по SPI или I2C.

   Примечание по заданию 8: Можете для работы с внешней платой брать сторонние библиотеки с ресурсов GitHub или туториалов, но главное чтобы в библиотеке были функции для коммуникации по I2C или SPI (желательно через библиотеку HAL).

9. Представьте в отчёте выполненные пункты 1-7 (или задание 8). Также в ответ приложите файлы main.c для STM-мастера и STM-slave, в котором реализованы работы с протоколами SPI/I2C.